

Socket Programming Project

QAP — Quiz Application Protocol

การสื่อสารแบบ Stateful สำหรับแอปพลิเคชันควิซผ่าน TCP

Web Browser Client

Python Terminal Client

วัตถุประสงค์ของโครงการ

- ✓ Application-Layer Protocol: ออกแบบโปรโตคอลเฉพาะทางสำหรับระบบ Quiz
- ✓ Multi-Client Support: รองรับการเล่นผ่าน Web Browser และ Terminal พร้อมกัน
- ✓ Stateful Management: ให้ Server จัดการ Session, ความคืบหน้า และคะแนนอย่างแม่นยำ
- ✓ Network Semantics: แสดงการทำงานของ Network Message และ Status Code ที่เข้าใจง่าย
- ✓ Advanced Features: รองรับ Flag Fragment, Image Upload และ Concurrent Users

System Architecture & Design

Core Characteristics

- ✓ **Client-Server Model:** แยกการทำงานส่วนระบบประมวลผลและผู้ใช้อย่างชัดเจน
- ✓ **Stateful Session:** จัดเก็บและติดตามสถานะผู้เล่นฝั่ง Server ตลอดเกม
- ✓ **Multi-Client Support:** รองรับการเชื่อมต่อพร้อมกันทั้ง Web Browser และ Terminal

Network & Execution Specs

- ✓ **Communication:** สื่อสารผ่าน TCP Socket Direct Connection Port 8080
- ✓ **Data Formats:** JSON Payload สำหรับข้อความ และ Base64 สำหรับไฟล์รูปภาพ
- ✓ **Concurrency:** ThreadPoolExecutor รองรับผู้ใช้งานสูงสุด 50 Workers

ทำไมเลือกใช้ TCP?

ความต้องการของ Quiz Application	คุณสมบัติของ TCP (Transmission Control Protocol)
คำถามและคำตอบต้องมาถึงครบถ้วน	Reliable Delivery: มีระบบตรวจสอบความถูกต้อง
ต้องอ่านข้อความตามลำดับที่ส่งมา	Ordered Byte Stream: ลำดับข้อมูลถูกต้องเสมอ
ต้องมีการรักษา Session ต่อเนื่อง	Connection-oriented: รักษาทางเดินข้อมูลระหว่างเล่น
ลดการเปิด/ปิด Connection บ่อยครั้ง	Persistent / Keep-alive: ลด Overhead ในการทำ Handshake

หากใช้ UDP จะต้องเขียนระบบ Acknowledgement, Retransmission และ Sequence Number ขึ้นมาเองทั้งหมด

QAP Message Format

```
METHOD /path HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
X-Session-ID:
Content-Length:

{
  "answer_id": 2
}
```

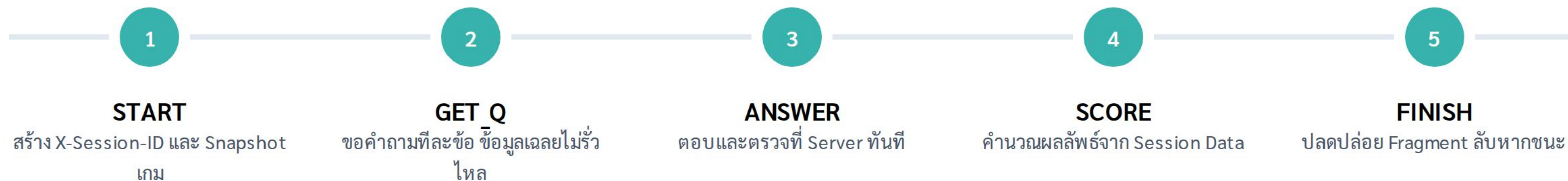
- ✓ Compatible Framing: ใช้รูปแบบ HTTP/1.1 เพื่อให้ Browser สื่อสารได้ง่าย
- ✓ X-Protocol Header: ระบุการใช้งาน Protocol QAP โดยเฉพาะ
- ✓ Session Tracking: ใช้ Custom Header สำหรับติดตามผู้เล่น
- ✓ Strict Termination: ส่วน Header จบด้วย \r\n\r\n เสมอ

QAP Methods & Status Codes

Method	Endpoint	หน้าที่หลัก
START	/api/start	สร้าง Session และทำ Snapshot ข้อมูลคำถาม
GET_Q	/api/question/{id}	ขอข้อมูลคำถาม (ไม่ส่งเฉลยใน Payload)
ANSWER	/api/answer/{id}	ส่งคำตอบเพื่อตรวจและเก็บคะแนน
SCORE	/api/score	สรุปผลคะแนนเมื่อตอบครบทุกข้อ

200 OK 401 SESSION_REQUIRED 409 CONFLICT (ตอบซ้ำ) 422 INCOMPLETE

Stateful Session & Server-Side Score



Security Note: Client ไม่สามารถส่งคะแนนปลอมได้ เพราะ Server เป็นผู้ถือครองสถานะ (State) และการคำนวณคะแนนทั้งหมด

Flag Fragment Chaining



Fragment Distribution

Server แบ่ง Flag ลับเป็นส่วนๆ ตามจำนวนข้อ
ตอบถูก 1 ข้อ ได้รับ 1 Fragment ใน Header



Index Ordering

ใช้ X-Flag-Fragment-Index เพื่อระบุตำแหน่งที่
ถูกต้อง ป้องกันปัญหาลำดับ Packet ของเครือข่าย



Secret Path

เมื่อรวม Fragment ครบ จะได้รหัสผ่านหรือ
Path ลับเพื่อใช้เปิดหน้าเฉลยพิเศษ

Error Cases & Anti-Cheat Logic

พฤติกรรม / การทดลอง	สถานะการตอบกลับจาก Server
เข้าถึง API โดยไม่มี X-Session-ID	401 SESSION_REQUIRED
พยายามตอบคำถามเดิมซ้ำเป็นครั้งที่สอง	409 CONFLICT
ขอสรุปคะแนน (SCORE) ทั้งที่ยังตอบไม่ครบ	422 INCOMPLETE
ส่งคะแนนปลอม 999 มาให้ Server	Server ไม่นำมาใช้ (คำนวณใหม่จาก History)
Upload รูปขนาด 10 MiB	413 PAYLOAD_TOO_LARGE

Source Code Core Implementation

Function / Module	ความรับผิดชอบ
start_session()	สร้าง Session ID ใหม่ และทำ Snapshot รายการคำถาม ณ เวลานั้น
handle_api_request()	Routing API ไปยังฟังก์ชันจัดการที่เหมาะสม
parse_request()	อ่าน Header และดึง Body ตามค่า Content-Length ที่ระบุ
assembled_flag()	(Client-side) เรียงลำดับและรวม Fragment ทั้งหมดที่ได้รับ
handle_client()	จัดการ Persistent Connection และ Multi-threading

CREATE & Image Upload Protocol

Upload Specifications

- ✓ Supported Formats: รองรับ PNG, JPEG และ GIF
- ✓ Size Limit: จำกัดขนาดไฟล์ไม่เกิน 5 MiB
- ✓ Base64 Encoding: แปลงภาพเป็นข้อความ เพื่อส่งใน JSON Payload เดียวกัน

Server Processing & Security

- ✓ Validation: ตรวจสอบ MIME Type และ Magic Bytes ป้องกันไฟล์อันตราย
- ✓ Storage: บันทึกลง Disk แบบ Atomic Save พร้อมตั้งชื่อไฟล์อัตโนมัติ

Overhead ของ Base64 เพิ่มขึ้น ~33% แต่ช่วยให้การจัดการ JSON รวมเป็นชุดเดียวได้สะดวก

Catalog & Session Snapshots

การจัดการข้อมูลคำถาม

ระบบรองรับการ **LIST** (ดูทั้งหมด) และ **DELETE** คำถามแบบเรียลไทม์ แต่มีการป้องกันไม่ให้ลบจนหมด (ลบข้อสุดท้ายไม่ได้)

แนวคิด Session Snapshot

เมื่อเริ่มเล่น (START) ระบบจะลือครายการคำถามไว้สำหรับ Session นั้นๆ แม้ Admin จะลบหรือเพิ่มคำถามระหว่างที่ผู้เล่นกำลังทำควิช จะไม่ส่งผลต่อเกมที่กำลังดำเนินอยู่

Persistent Connection Efficiency

Connection-close (ทั่วไป)

12 TCP Connections

Persistent (QAP)

1

ทำไมถึงเร็วขึ้น?

- ✓ ไม่ต้องทำ TCP Handshake (SYN-ACK) ทุก Request
- ✓ ลด Overhead ของระบบเครือข่าย
- ✓ ใช้ Content-Length ในการแบ่ง Message บน Stream

ตัวอย่าง 1 เกม (5 ข้อ):

- ✓ START (1) + GET_Q (5) + ANSWER (5) + SCORE (1)
- ✓ รวม 12 Requests บนการเชื่อมต่อเดียว

Concurrent Load Test Performance



50 Workers

ใช้ ThreadPoolExecutor รองรับการร้องขอที่เข้ามาพร้อมกันอย่างมีประสิทธิภาพ



Latency P95

วัดผลความหน่วงเพื่อตรวจสอบความเสถียรภายใต้ภาระงานสูง (Stress Test)



Session Isolation

ใช้ Lock แยกตาม Session ป้องกันสภาวะการแย่งชิงข้อมูล (Race Condition)

```
python stress_test.py --clients 10 25 50
```


สรุปและข้อจำกัดของระบบ

ความสำเร็จของโครงการ

- ✓ โพรโทคอล QAP ทำงานได้สมบูรณ์แบบบน TCP
- ✓ จัดการ Stateful Session และ Score ได้อย่างแม่นยำ
- ✓ รองรับ Image Upload และการจัดการคำถาม
- ✓ ระบบมีความทนทานต่อการรบกวนข้อมูล (Anti-Cheat)

ข้อจำกัดและงานในอนาคต

- ✓ เพิ่มการเข้ารหัสข้อมูลด้วย TLS/SSL
- ✓ นำ Persistent DB มาใช้แทน Memory
- ✓ เพิ่มระบบ Authentication / Rate Limiting
- ✓ ขยายผลสู่ระบบ Quiz แบบเรียลไทม์ (Websocket)

Questions?

Thank you for your attention